TRANSACTION-CONCURRENCY-CONTROL

트랜잭션과 병행제어

연관 기출 · 128-4-5, 130-1-2, 129-1-6

한눈에 이해

두 직원이 재고 10개를 각각 읽고 한 개씩 판매합니다. 두 사람이 모두 재고를 9로 저장하면 한 건의 차감이 사라집니다. 트랜잭션은 한 업무를 묶는 단위이고, 병행제어는 이런 동시 작업이 서로 영향을 주는 방식을 조정합니다.

정의와 핵심 원리

트랜잭션은 원자성·일관성·격리성·지속성(ACID)을 갖는 논리적 작업 단위이며 병행제어는 동시 실행의 충돌을 조정합니다.

• 원자성은 한 작업의 변경을 모두 반영하거나 모두 취소한다는 뜻입니다. 지속성은 커밋한 변경을 장애 뒤에도 복구할 수 있다는 뜻입니다. 일관성은 업무 제약을 지키는 상태로 바뀌는 성질이며, 격리성은 동시 실행에서 허용하는 간섭을 제한합니다. ACID의 일관성과 CAP의 일관성은 각각 가리키는 성질이 다릅니다.

• 2단계 잠금은 먼저 잠금을 얻고, 하나라도 풀기 시작하면 새 잠금을 얻지 않습니다. 이렇게 충돌 직렬가능성을 확보하지만 대기와 교착이 생길 수 있습니다. 타임스탬프 순서는 충돌 작업의 시간 순서를 강제합니다. 낙관적 제어는 커밋 전에 충돌을 검사해 실패 시 재시도합니다.

• 다중 버전 병행 제어(MVCC)는 행의 여러 버전을 두고 조회 시점에 보이는 버전의 집합(스냅샷)을 사용해 읽기와 쓰기 사이의 대기를 줄입니다. 갱신 손실(lost update)이나 쓰기 왜곡(write skew)을 막을 수 있는지는 격리 수준과 DBMS 구현에 따라 달라집니다. 트랜잭션을 짧게 유지하고, 잠금 순서와 교착 재시도 정책을 정합니다.

구조·비교·판단

구분	원리·특징	적용·주의
잠금 기반	충돌 전에 자원 접근 조정	대기·교착
낙관 검증	충돌을 나중에 검출	충돌이 많으면 재시도 증가
MVCC	버전별 snapshot 읽기	저장량·정리·격리 설정

적용 예제

재고 lost update

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 재고 10을 A와 B가 읽고 각각 1개 판매합니다.

1. A가 9를 쓰고 B도 9를 쓰면 두 판매 뒤 재고가 9가 되는 lost update입니다.

2. UPDATE 재고 SET 수량=수량-1 WHERE 상품=1 AND 수량>0처럼 DB 안에서 현재 값에 1을 빼고, 실제로 갱신된 행 수를 확인합니다.

3. 업무가 여러 행에 걸치면 동일 트랜잭션으로 묶고 적절한 격리·잠금 또는 충돌 재시도를 설계합니다.

결과·해석: 두 판매가 정상 커밋하면 재고 8이 되어야 합니다.

전제·주의: 원자 증가/감소 하나의 해결책을 임의의 복잡한 업무 불변식에 그대로 일반화하지 않습니다.

답안 골격과 혼동 방지

ACID와 병행제어 필요성 → 대표 이상 현상 → 잠금/시간순서/검증/MVCC → 재고 사례 → 교착·기아·재시도 대책

- 격리성이 무조건 모든 실행을 실제 한 줄로 세운다는 뜻은 아닙니다.
- 낮은 격리와 업무 코드의 읽기-쓰기 조합을 함께 봅니다.

내용을 가리고 회상

1. lost update를 두 트랜잭션의 순서로 보일 수 있습니까?

2. MVCC와 잠금은 반드시 서로 배타적인 대안입니까?

3. 낙관 제어는 충돌이 잦을 때 어떤 비용이 생깁니까?

적용 기준과 근거

ACID·MVCC·격리 구현은 공식 문서를 확인했습니다. 일반 2PL/타임스탬프/낙관 검증의 상세 알고리즘 전체는 추가 교재 확인 범위입니다.

내용 검토 상태: 부분 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- PostgreSQL 18: Concurrency Control - PostgreSQL Global Development Group; PostgreSQL 18; 확인 2026-11-05.
- PostgreSQL 16: Transaction Isolation - PostgreSQL Global Development Group; PostgreSQL 16; 확인 2026-11-05.